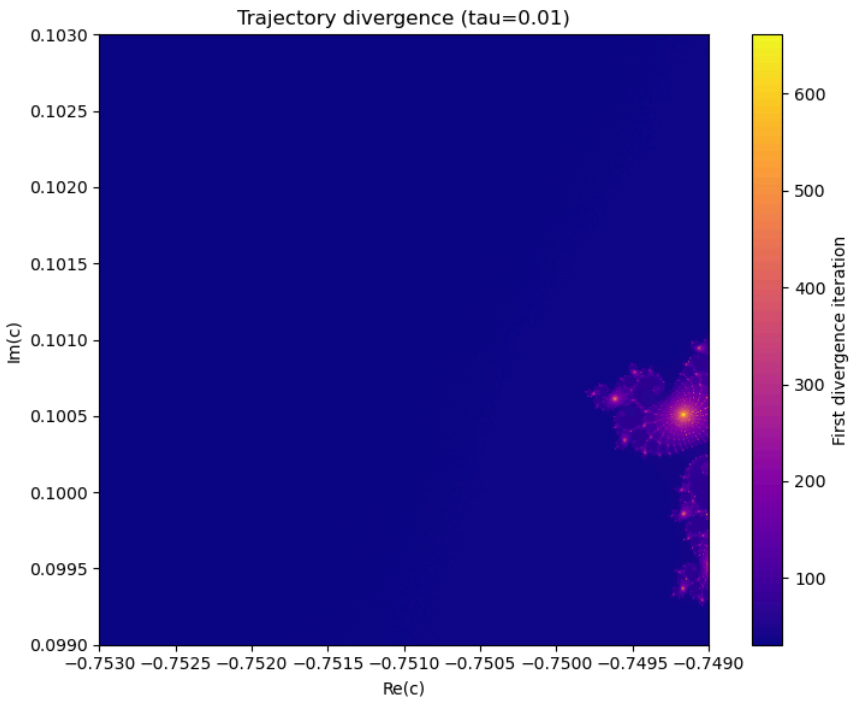
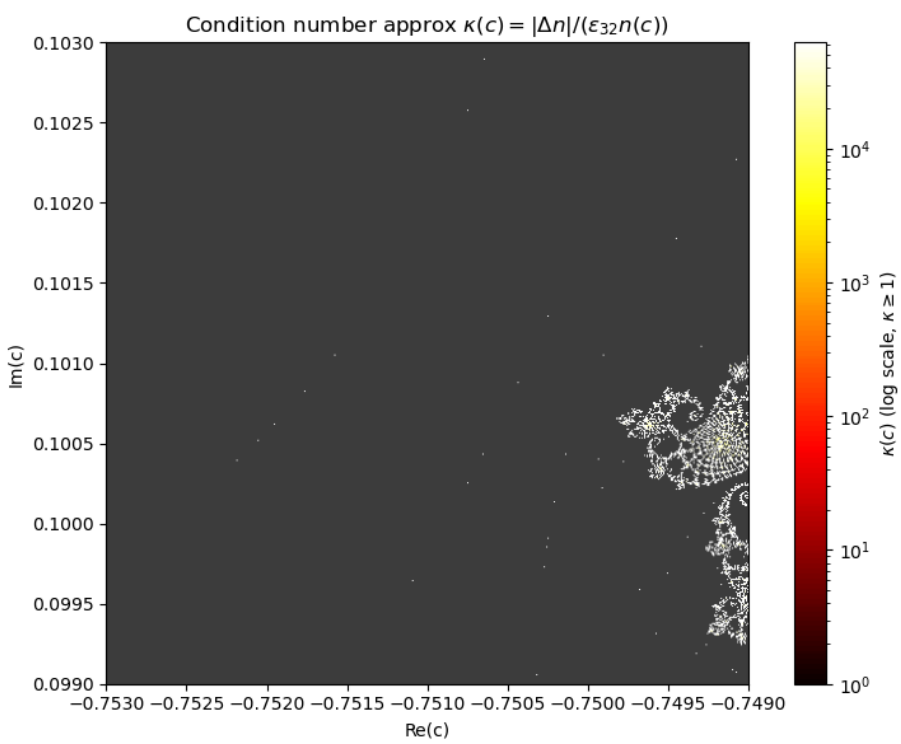
Section 1: Numerical accuracy

Divergence map (m1)



Sensitivity map (m2)



Do the M1 and M2 maps agree spatially? Where is sensitivity highest?
The M1 and M2 maps agree quite well spatially. Both highlight the same filament-like structure near the right side of the plotted region. This means that the areas where the float32 and float64 trajectories diverge late are also the areas where the escape count is most sensitive to small perturbations in c. The highest sensitivity is found along the fine boundary/filament structure close to the Mandelbrot set boundary.

Section 2: Testing and Documentation

Pytest -v (zoom in)

L09: M1: Test Suite Results

Test Results

```
(nsc2026) C:\Users\chris\Documents\AVS\8sem\NumericalScientificComputing\Mandelbrot>pytest -v
============================================================= test session starts ==============================================================
platform win32 -- Python 3.11.14, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\chris\miniforge3\envs\nsc2026\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\chris\Documents\AVS\8sem\NumericalScientificComputing\Mandelbrot
plugins: cov-7.1.0
collected 11 items

test_mandelbrot.py::test_naive_point_known_values[origin_stays_bounded] PASSED                                                          [  9%]
test_mandelbrot.py::test_naive_point_known_values[far_outside_right_escapes_immediately] PASSED                                         [ 18%]
test_mandelbrot.py::test_naive_point_known_values[far_outside_left_escapes_immediately] PASSED                                          [ 27%]
test_mandelbrot.py::test_naive_point_known_values[hits_radius_two_then_escapes_next_step] PASSED                                        [ 36%]
test_mandelbrot.py::test_numba_point_known_values[origin_stays_bounded] PASSED                                                          [ 45%]
test_mandelbrot.py::test_numba_point_known_values[far_outside_right_escapes_immediately] PASSED                                         [ 54%]
test_mandelbrot.py::test_numba_point_known_values[far_outside_left_escapes_immediately] PASSED                                          [ 63%]
test_mandelbrot.py::test_numba_point_known_values[hits_radius_two_then_escapes_next_step] PASSED                                        [ 72%]
test_mandelbrot.py::test_numba_grid_matches_naive_on_small_float64_grid PASSED                                                          [ 81%]
test_mandelbrot.py::test_multiprocessing_parallel_matches_serial_on_small_grid PASSED                                                   [ 90%]
test_mandelbrot.py::test_build_chunks_covers_all_rows_once PASSED                                                                        [100%]

============================================================== 11 passed in 9.98s ==============================================================
```

Pytest --cov (zoom in)

```
(nsc2026) C:\Users\chris\Documents\AVS\8sem\NumericalScientificComputing\Mandelbrot>pytest --cov=. -v
============================================================================= test session starts ==============================================================
platform win32 -- Python 3.11.14, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\chris\miniforge3\envs\nsc2026\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\chris\Documents\AVS\8sem\NumericalScientificComputing\Mandelbrot
plugins: cov-7.1.0
collected 11 items

test_mandelbrot.py::test_naive_point_known_values[origin_stays_bounded] PASSED                                                          [  9%]
test_mandelbrot.py::test_naive_point_known_values[far_outside_right_escapes_immediately] PASSED                                         [ 18%]
test_mandelbrot.py::test_naive_point_known_values[far_outside_left_escapes_immediately] PASSED                                          [ 27%]
test_mandelbrot.py::test_naive_point_known_values[hits_radius_two_then_escapes_next_step] PASSED                                        [ 36%]
test_mandelbrot.py::test_numba_point_known_values[origin_stays_bounded] PASSED                                                          [ 45%]
test_mandelbrot.py::test_numba_point_known_values[far_outside_right_escapes_immediately] PASSED                                         [ 54%]
test_mandelbrot.py::test_numba_point_known_values[far_outside_left_escapes_immediately] PASSED                                          [ 63%]
test_mandelbrot.py::test_numba_point_known_values[hits_radius_two_then_escapes_next_step] PASSED                                        [ 72%]
test_mandelbrot.py::test_numba_grid_matches_naive_on_small_float64_grid PASSED                                                          [ 81%]
test_mandelbrot.py::test_multiprocessing_parallel_matches_serial_on_small_grid PASSED                                                   [ 90%]
test_mandelbrot.py::test_build_chunks_covers_all_rows_once PASSED                                                                        [100%]

================================================================ tests coverage ================================================================
_____________________________________________________________ coverage: platform win32, python 3.11.14-final-0 _____________________________________________________

Name                        Stmts   Miss  Cover
-----------------------------------------------
Workshop1\test_script.py       12      0   100%
Workshop2\test_dask.py          9      6    33%
conftest.py                    17      3    82%
mandelbrot_mp.py               90     58    36%
mandelbrot_naive.py            48     19    60%
mandelbrot_numba.py            51     41    20%
test_mandelbrot.py             34      0   100%
-----------------------------------------------
TOTAL                         261    127    51%
============================================================== 11 passed in 8.10s ==============================================================
```

Which implementations are covered? Did writing tests reveal anything unexpected?
I covered the naive, Numba, and multiprocessing implementations. The tests focus on the core Mandelbrot computation: known tests for the pixel function, naive-vs-numba comparison on a small grid, and serial-vs-parallel comparison for multiprocessing. Writing the tests revealed a few issues: some implementations are not directly comparable because they use slightly different grid/iteration semantics, and the naive version also depends on the ‘line_profiler’, which made clean testing more fragile for me without refactoring.

Section 3: GPU performance

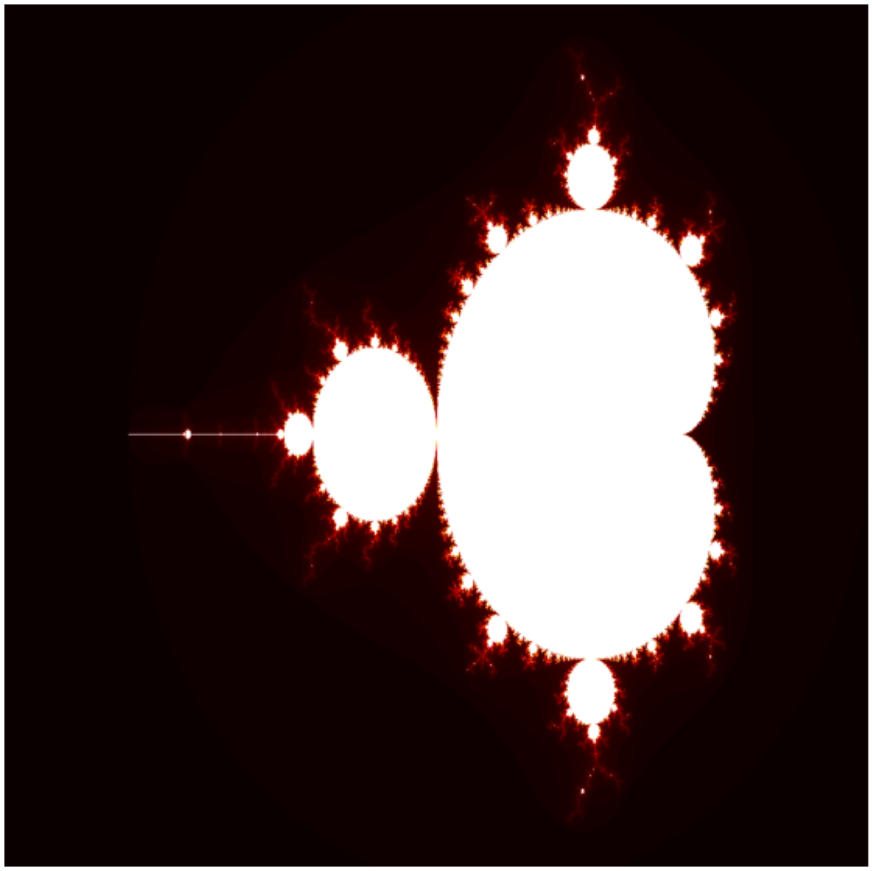
GPU float32 runtime and image at N=1024

L10: M1: Mandelbrot GPU Implementation

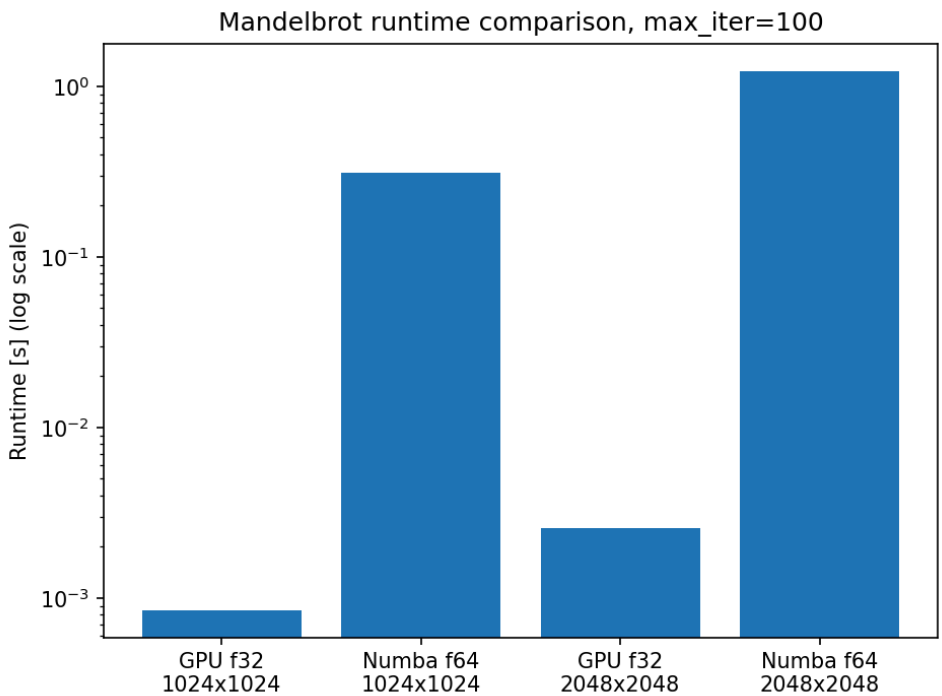
Timing Results

```
GPU 1024x1024: 3.7 ms
GPU 2048x2048: 6.3 ms
```

A corresponding mandelbrot_gpu_m1.png image is saved.



Float32 vs float64 (numba) speed ratio



L10: M2: GPU f32 vs numba f64 Performance Comparison

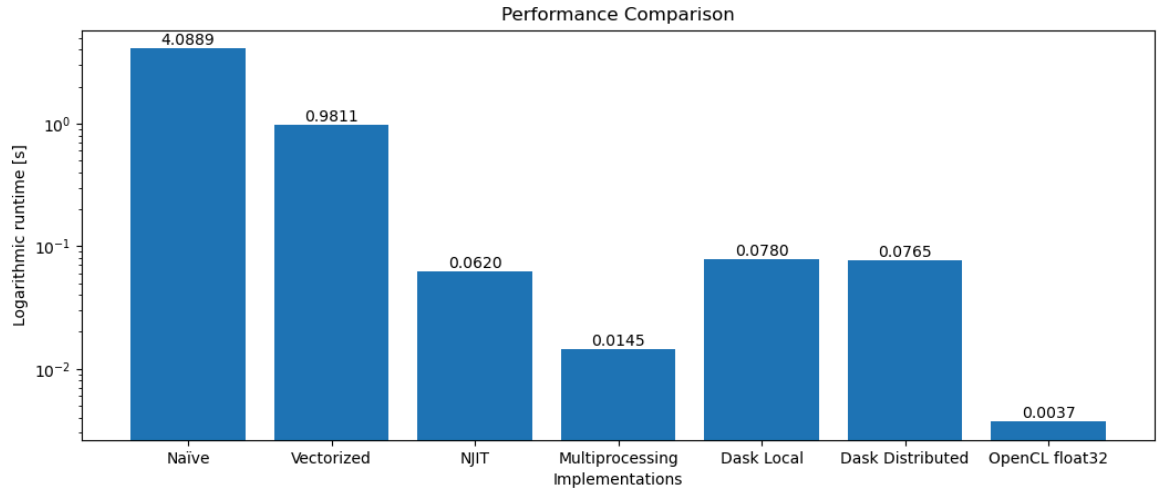
Timing Results

```
GPU f32 1024x1024: 0.8 ms median over 3 runs
Numba f64 1024x1024: 311.0 ms
Speedup vs Numba f64 at N=1024: 367.26x
GPU f32 2048x2048: 2.6 ms median over 3 runs
Numba f64 2048x2048: 1234.0 ms
Speedup vs Numba f64 at N=2048: 477.06x

N           GPU f32 [s]   Numba f64 [s]    Numba f64 / GPU f32  Numba f32 [s]
-----------------------------------------------------------------------------
1024          0.000847       0.311000               367.26x       0.071000
2048          0.002587       1.234000               477.06x       0.285000
```

Added numba f32 just for comparison.

Combined benchmark bar chart



L10: Final Benchmarking Results

Benchmarking Results

```
Speedup Table at 1024x1024 resolution, max_iter=100

| Implementation | Time (s) | Speedup |
|----------------|----------|---------|
Naive Python     4.0889 s,   speedup=1.00x
Numpy Vectorized 0.9811 s,   speedup=4.17x
Numba (@njit)    0.0620 s,   speedup=65.95x
Multiprocessing  0.0145 s,   speedup=281.99x
Dask(6 workers, 32 chunks) 0.1276 s, speedup=32.04x
Dask(6 workers, 12 chunks) 0.0780 s, speedup=52.42x
Dask Distributed (4 workers, 12 chunks) 0.0765 s, speedup=53.78x
GPU F32(1024x1024) 0.0037 s,   speedup=1105.11x
GPU F32(2048x2048) 0.0063 s,   speedup=649.03x
```

Brief comment: where does GPU sit in the overall speedup progression? Any surprising result?

GPU is significantly faster than the other implementations, achieving a speedup of 1105x faster than the naive python implementation on 1024x1024 resolution. It was expected that the GPU implementation would be significantly faster.